

결 과 보 고 서

과제명: An Efficient CNN Accelerator with P3LQR and
a Streaming FC Architecture

팀 명: 부산의 인재

경북대학교 IDEC 2025 창의 회로설계 챌린지

1. 과제 개요

CNN(Convolutional Neural Network)은 이미지의 공간적 특징을 추출하는 딥러닝 모델로, 오늘날 AI 기반 이미지 인식의 핵심 기술이다. CNN은 대규모 MAC(Multiply-Accumulate) 연산을 통해 이미지를 분석한다. 이러한 연산 집약적인 구조는 높은 정확도를 가능하게 하지만, 동시에 막대한 계산량을 요구한다. 특히 전력 공급이 제한적이고 연산 자원이 부족한 엣지 디바이스 환경에서는 연산 오버헤드가 하드웨어의 전력 소비와 처리 속도에 직접적인 병목으로 작용하므로, 알고리즘 및 하드웨어 수준의 최적화가 필수적이다.

본 프로젝트에서 최적화하고자 하는 Baseline은 두 개의 5×5 Convolution layer와 하나의 FC(Fully Connected) layer로 구성된 CNN 가속기로, MNIST 데이터셋 (0부터 9까지의 숫자 이미지)을 분류하는데 사용된다. 이 가속기를 개선하기 위해 세 가지 최적화 기법을 제안한다:

- P3LQR (Power-of-Two Parameter Pruning with Log Quantization of Residual) Algorithm: Pruning은 네트워크에서 중요도가 낮은 가중치를 제거하여 모델 크기를 줄이는 기법으로, 일반적으로 절댓값이 작은 가중치를 제거하는 방식으로 수행된다. 이를 통해 메모리 사용량과 연산량을 줄여 전력 효율을 개선하고 추론 속도를 향상할 수 있다. Log Quantization은 가중치를 로그 도메인에서 표현하는 기법으로, MAC 연산의 곱셈을 Shift 연산으로 대체할 수 있다는 장점이 있다. 이를 통해 전력 소모가 큰 곱셈기를 Shift 모듈로 치환하여 에너지 효율을 크게 높일 수 있다. 그러나 Log Quantization은 작은 가중치는 정밀하게 표현하지만 큰 가중치는 상대적으로 덜 정밀하게 표현하는 특성이 있어, 단순히 Pruning과 병행하면 정확도가 크게 저하될 수 있다. 본 프로젝트에서는 이 문제를 해결하기 위해 Pruning과 Log Quantization을 효과적으로 결합한 P3LQR 기법을 새롭게 제안한다.
- Streaming FC Architecture: Baseline의 FC layer는 모든 입력을 메모리에 저장했다가 연산을 시작하는 구조로 설계되어 있어, 메모리 사용량이 많고 latency가 크다는 단점이 있다. 이를 해결하기 위해 FC layer를 입력 스트리밍 기반 구조로 개선하여 전력 및 면적 효율을 개선하는 동시에 추론 속도를 향상한다.
- Convolution Layer Pipelining: Baseline은 모듈 단위로 파이프라이닝 되어 있으나, 모듈 간 critical path delay가 불균일해 비효율적이다. 이를 개선하기 위해 각 모듈의 delay를 분석하고, 상대적으로 latency가 큰 conv1_calc, conv2_calc 모듈 내부를 추가적으로 파이프라이닝함으로써 동작 주파수를 높이고 시스템 전반의 성능을 개선한다.

제안한 최적화 기법의 효과를 정량적으로 평가하기 위해 TOPS/W (Tera Operations per Watt)를 사용하였다. TOPS/W는 다음과 같이 정의된다.

$$TOPS/W = \frac{(\# of Operations) \times (frequency)}{10^{12} \times (power)}$$

여기서 CNN의 주요 연산은 MAC 연산이며, MAC 연산은 곱셈 1회와 덧셈 1회로 구성되므로,

$$\# of Operations = 2 \times (\# of MAC)$$

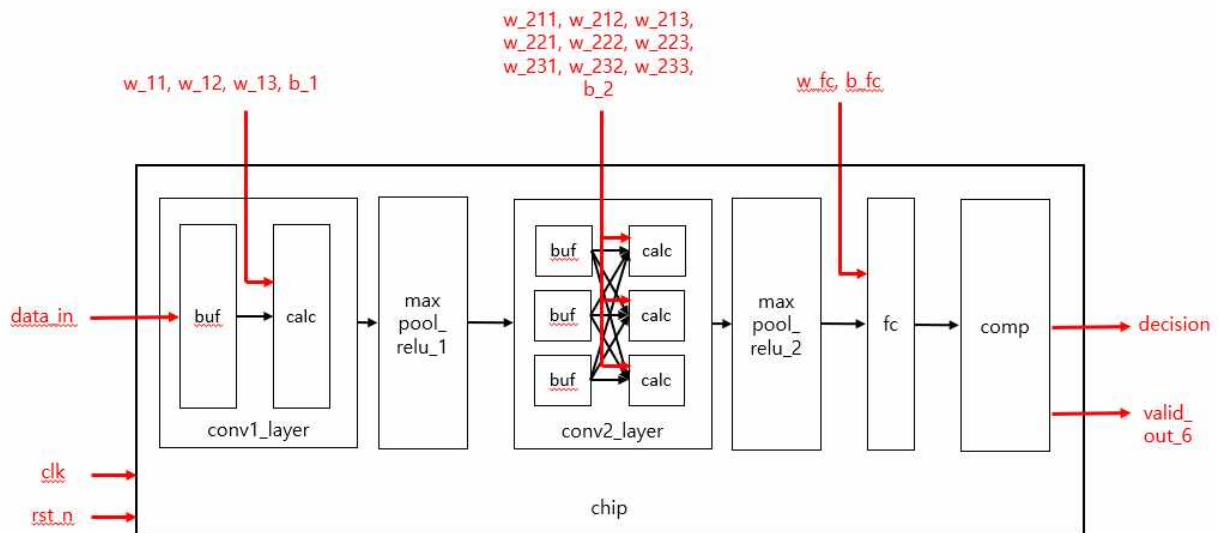
으로 계산할 수 있다.

이 방법으로 계산한 결과 Baseline의 TOPS/W는 0.989였으며 Proposed design은 19.496으로 1871% 향상되었다. (Pruning 전 Dense 모델의 총 MAC 유닛 수(348개)를 기준으로 한 Dense-equivalent 연산량으로 계산한 수치). 또한 추론 시간이 68.1% 단축되었고, 면적은 55.1% 감소하였다. 따라서 본 프로젝트에서 제안한 P3LQR 알고리즘, Streaming FC Architecture, Convolution Layer Pipelining은 CNN 하드웨어의 전력 소모, 추론 속도, 그리고 하드웨어 면적의 세 가지 지표(PPA)를 모두 개선하는 효과적인 방법임을 확인하였다.

2. 설계 목표

2.1 Baseline 분석

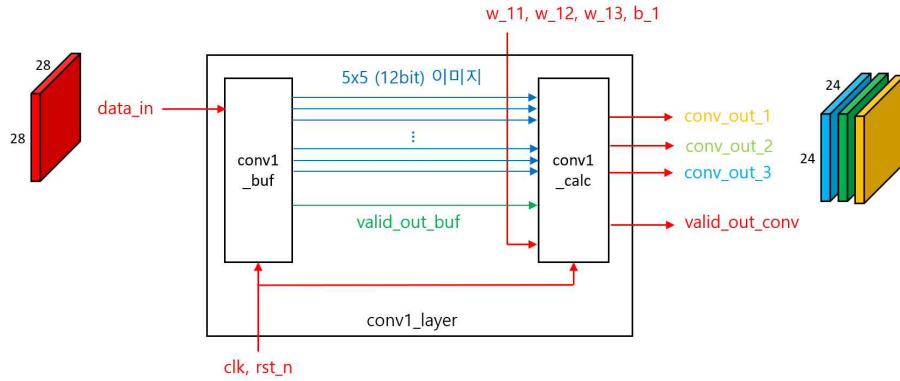
본 프로젝트에서 최적화하고자 하는 Baseline은 두 개의 5×5 Convolution layer와 하나의 FC layer로 구성된 CNN 가속기로, MNIST 데이터셋 (0부터 9까지의 숫자 이미지)을 분류하는 데 사용된다. 각 Convolution layer 뒤에는 Max Pooling과 ReLU 활성화 함수가 연결되며, 출력단에서 비교기가 FC layer에서 생성된 10개의 출력을 입력받아 최댓값을 선택함으로써, 입력 이미지가 어느 숫자에 해당하는지 결정된다. 전체적인 Baseline 모델의 구조는 [그림 1]에 나타내었다.



[그림 1] Baseline 구조

2.1.1 conv1_layer

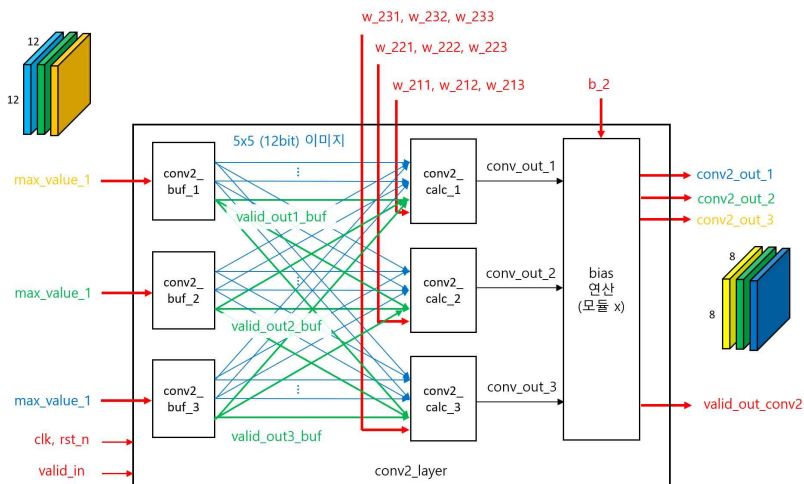
conv1_layer는 입력으로 주어진 $28 \times 28 \times 1$ 크기의 MNIST 이미지를 받아 5×5 convolution 연산을 수행하고, $24 \times 24 \times 3$ 크기의 Activation을 출력한다. 이 모듈은 두 개의 하위 모듈 conv1_buf와 conv1_calc로 구성된다. conv1_buf는 매 클럭 당 한 픽셀이 순차적으로 입력되는 스트리밍 데이터를 내부 버퍼에 저장하고, convolution 연산에 필요한 5×5 크기의 이미지를 병렬로 출력하는 슬라이딩 윈도우 역할을 수행한다. conv1_calc 모듈은 conv1_buf에서 전달된 5×5 이미지와 5×5 가중치에 대한 MAC 연산을 수행하고, 바이어스를 더하여 convolution 결과를 출력한다.



[그림 2] conv1_layer

2.1.2 conv2_layer

conv2_layer는 $12 \times 12 \times 3$ 크기의 Activation을 입력으로 받아 5×5 convolution 연산을 수행하고 $8 \times 8 \times 3$ 크기의 Activation을 출력한다. 이 모듈은 conv1_layer와 동일하게 두 개의 하위 모듈 conv2_buf와 conv2_calc로 구성된다. conv2_layer에서는 입력이 세 채널이므로 각 채널마다 독립적으로 동작하는 conv2_buf 모듈이 3개 존재한다. 각 buf 모듈은 해당하는 채널의 입력을 받아 슬라이딩 윈도우 역할을 수행한다. conv2_calc 모듈 역시 세 개로 구성되며, 각 모듈은 3개의 buf 모듈에서 전달된 5×5 크기의 입력을 받아 총 $5 \times 5 \times 3$ 크기의 입력 윈도우를 형성한다. 이 입력과 동일한 크기의 가중치와 MAC 연산을 수행하고, 바이어스를 더하여 convolution 결과를 출력한다.



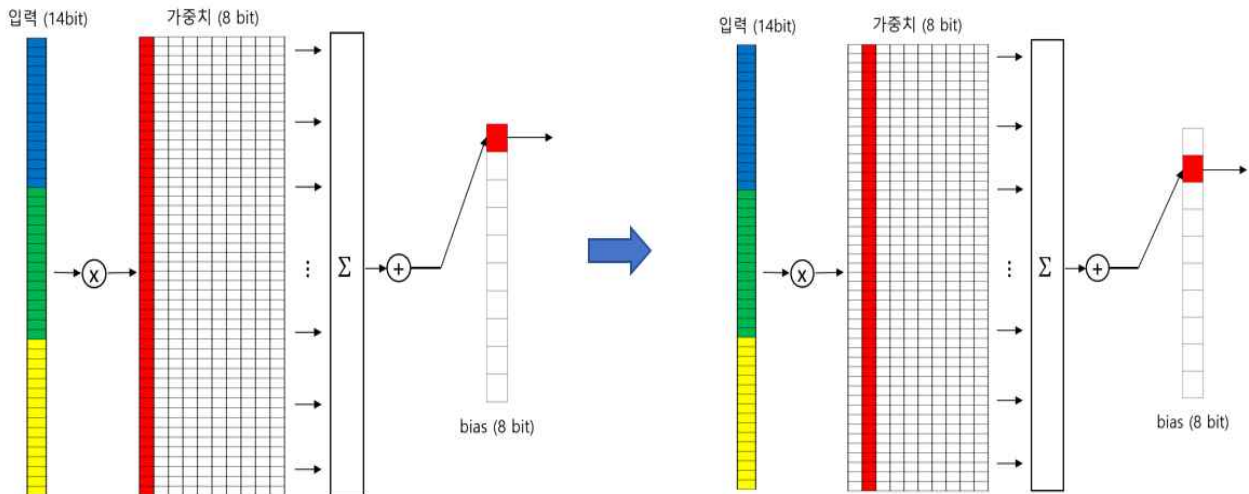
[그림 3] conv2_layer

2.1.3 Maxpool_relu

Maxpool_relu는 conv1_layer와 conv2_layer 뒤에 배치되어 2×2 Max pooling과 ReLU 활성화 함수 역할을 수행한다. 이 모듈은 연속적으로 들어오는 입력 두 개 중 더 큰 값만 내부 line 버퍼에 저장한다. 짝수 행의 저장이 끝나고 홀수 행이 입력되면, line 버퍼에 저장된 값과의 비교 연산을 통해 2×2 윈도우 내에서 최댓값을 계산한다. 계산된 값은 음수일 경우 0으로 변환되고, 양수일 경우 그대로 출력된다. 홀수 행과의 비교 연산이 끝나면 다음 짝수 행이 다시 line 버퍼에 저장된다.

2.1.4 FC layer

FC layer는 스트리밍 방식으로 들어오는 $4 \times 4 \times 3 = 48$ 크기의 입력을 받아, 최종적으로 10개의 출력을 생성하는 역할을 한다. 이를 위해 480개의 가중치(48×10)와 10개의 바이어스가 사용된다. 입력은 먼저 48개의 입력 버퍼에 순차적으로 저장되며, 버퍼가 가득 차면 연산이 시작된다. 이때 가중치 버퍼에 저장된 480개의 가중치 중 48개씩을 순차적으로 불러와, 총 10번의 연산을 수행한다. 각 연산은 48개의 입력과 가중치의 MAC 연산 및 바이어스를 더하는 과정으로 이루어지며, 이를 통해 10개의 출력이 순차적으로 완성된다. FC layer의 구체적인 연산 과정은 [그림 4]에 자세히 나타내었다.



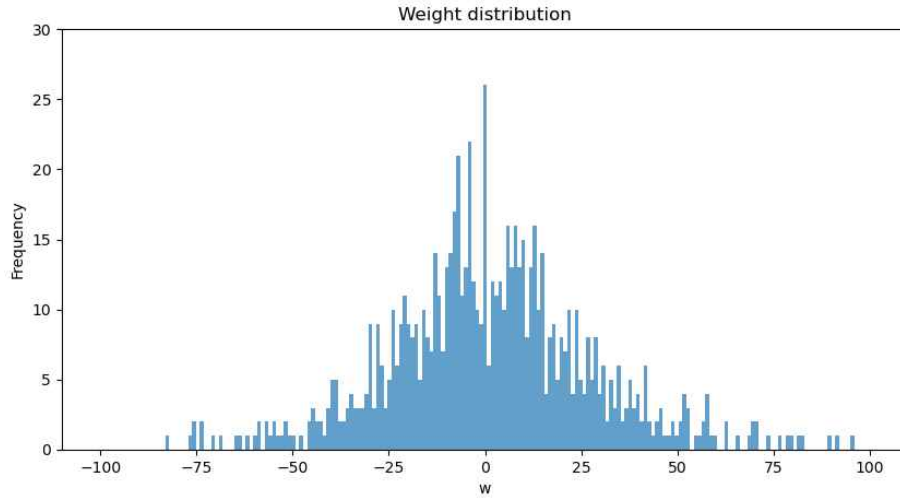
[그림 4] FC layer의 연산 과정

2.1.5 Comparator

Comparator는 FC layer에서 생성된 10개의 출력값을 순차적으로 입력받아 내부 버퍼에 저장한 뒤, 이들 중 최댓값을 선택하여 입력 이미지가 어느 숫자에 해당하는지를 결정한다. 비교 과정은 총 4개의 사이클로 이루어진다. 첫 번째 사이클에서는 10개의 입력을 2개씩 짝지어 총 5쌍을 비교하고, 두 번째 사이클에서는 남은 5개의 값 중 4개를 비교하여 2개의 후보를 남긴다. 세 번째 사이클에서는 이 2개의 값을 비교해 하나의 후보를 얻고, 마지막 네 번째 사이클에서 최종적으로 두 값을 비교하여 전체 10개 중 최댓값을 선택한다. 이 과정을 통해 네트워크의 최종 분류 결과가 결정된다.

2.2 최적화 전략

Baseline 디자인의 구조를 분석한 결과, 세 가지의 주요한 비효율성이 확인되었다. 첫째, 네트워크 내 많은 가중치가 0에 가까운 값을 가지고 있어 연산 결과에 미치는 영향이 무시할 만큼 작음에도 메모리에 저장되고 연산에 사용되고 있다([그림 5]). 이를 해결하기 위해 pruning을 적용하여 메모리 용량을 줄이고, 이후 발생한 weight sparsity를 하드웨어 설계에 반영함으로써 PPA 개선을 추구하고자 하였다. 또한 가중치에 Log Quantization을 적용함으로써, 전력 소모와 면적이 큰 곱셈기를 시프트 모듈로 치환하고 전력 및 면적 효율을 개선하였다.



[그림 5] 가중치 분포

둘째, Baseline의 FC layer는 입력이 모두 들어온 후 연산을 시작하는 구조로 설계되어 있어, 불필요하게 큰 버퍼와 긴 latency가 발생하였다. 따라서 FC layer를 입력 스트리밍 기반 구조로 최적화하여 PPA를 개선하는 것을 목표로 하였다.

마지막으로, Baseline에서 MAC 연산을 수행하는 모듈의 latency가 다른 모듈에 비해 훨씬 길어 비효율적인 파이프라이닝이 되었다. 이를 개선하기 위해, 최적화된 각 모듈의 worst slack을 분석한 후, latency가 가장 큰 MAC 연산 모듈을 파이프라이닝하여 동작 주파수를 개선하고, 추론 속도를 향상하였다.

3. 설계 및 시뮬레이션

3.1 최적화 방법

3.1.1 P3LQR (Power-of-Two Parameter Pruning with Log Quantization of Residual)

가중치는 신경망에서 연결의 강도를 의미하며, 가중치 값이 작은 연결일수록 중요도가 낮아 해당 연결을 제거해도 정확도에 미치는 영향은 크지 않다. Pruning은 이러한 중요도가 낮은 연결을 제거하여 정확도를 유지하면서 모델을 경량화하는 기법이다. 이를 통해 모델 크기가 줄어들어 메모리 사용량이 감소하고, 연산량이 줄어 추론 속도가 빨라지며 전력 소모 역시 절감된다. 또한 과적합을 완화하여 일반화 성능을 개선하는 데에도 기여할 수 있다. 본 프로젝트에서는 각 layer 별로 threshold 값을 설정하고, 그보다 절댓값이 작은 가중치를 제거하는 방식을 적용한다. 이 과정에서 정확도를 최대한 유지하면서 동시에 가능한 많은 가중치를 제거할 수 있도록 적절한 threshold를 설정하는 것이 중요하다.

한편, Log Quantization은 CNN의 핵심 연산인 MAC (Multiply-Accumulate) 연산을 단순화하기 위해, 신경망의 가중치를 로그 도메인에서 표현하는 양자화 방식이다. 일반적으로 MAC 연산은

$$y = \sum_{i=1}^n w_i \cdot x_i$$

로 표현된다. 여기서 가중치 w 를 Log Quantization 하는 수식은 아래와 같다.

$$\tilde{w} = \begin{cases} 0, & |w| = 0 \\ \text{Clip}(\lfloor \log_2 |w| \rfloor, \min, \max), & |w| \neq 0 \end{cases}$$

위 식에서 $\lfloor \cdot \rfloor$ 은 floor 함수이고, $\text{Clip}(\alpha, \min, \max)$ 는 값 α 를 $\min$ 과 $\max$ 사이로 제한하는 연산이다. $|w| = 0$ 인 경우에 대한 식은, 로그 연산에서 $\log_2(0)$ 이 정의되지 않기 때문에 필요하다. 양자화된 가중치의 부호는

$$s = \begin{cases} 0, & w \geq 0 \\ 1, & w < 0 \end{cases}$$

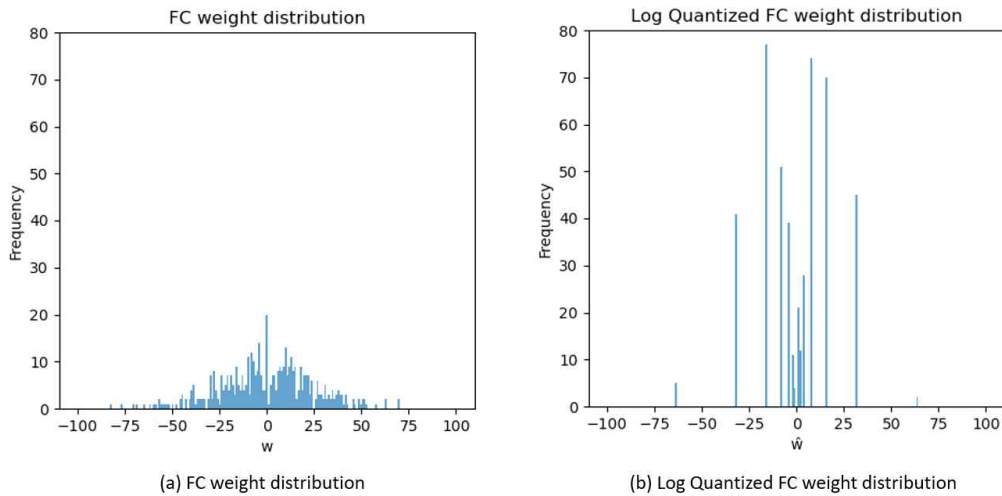
로 저장한다. 최종적으로 복원된 가중치는

$$\hat{w} = (-1)^s \cdot 2^{\tilde{w}}$$

로 표현된다. 이를 MAC 연산에 적용하면

$$y \approx \sum_{i=1}^n \hat{w}_i \cdot x_i = \sum_{i=1}^n (-1)^{s_i} \cdot 2^{\tilde{w}_i} \cdot x_i = \sum_{i=1}^n (-1)^{s_i} \cdot (x_i \ll \tilde{w}_i)$$

와 같이 곱셈 연산이 shift 연산으로 단순화된다. [그림 6]는 Baseline의 FC layer 가중치 값들에 Log Quantization을 적용한 결과를 보여준다.



[그림 6] Log Quantization을 적용한 FC layer의 가중치

CNN 모델에 Log Quantization을 적용하면 전력 소모가 큰 곱셈기를 Shift 모듈로 대체할 수 있어 회로 면적이 줄고 연산 속도가 빨라진다. 또한 가중치를 부호와 지수만으로 표현할 수 있기 때문에 메모리 사용량과 대역폭 요구량도 감소한다. 그러나 [그림 6]에서 확인할 수 있듯이 Log Quantization은 0 근처의 절댓값이 작은 가중치를 정밀하게 표현하고, 절댓값이 큰 가중치는 상대적으로 덜 정밀하게 표현하는 특성이 있어 Pruning과 동시에 적용하기 어렵다. 이는 Pruning 후 남게 되는 큰 가중치들이 Log Quantization에서는 충분히 정밀하게 표현되지 못하기 때문이다. 이러한 상충 효과로 인해 두 기법을 단순히 병행하면 정확도가 크게 저하될 수 있다. 따라서 본 프로젝트에서는 Pruning과 Log Quantization을 효과적으로 결합한 P3LQR (Power-of-Two Parameter Pruning with Log Quantization of Residual) 기법을 제안한다.

P3LQR은 두 단계로 진행된다. 첫 번째 단계는 네트워크에서 절댓값이 작은 가중치를 2^p 을 threshold로 하여 제거하는 Power-of-Two Parameter Pruning이다. 두 번째 단계는 pruning 후 남은 non-zero 가중치의 residual을 로그 도메인으로 양자화하는 Log Quantization of Residual이다.

3.1.1.1 Power-of-Two Parameter Pruning

첫 번째 단계에서 threshold는 2의 거듭제곱으로 정의된다. 각 layer마다 pruning exponent p 를 선택하고, threshold를 2^p 으로 둔다. Power-of-Two Parameter Pruning은 다음과 같이 정의된다.

$$P_p(w) = \begin{cases} 0, & |w| < 2^p \\ w, & |w| \geq 2^p \end{cases}$$

즉, threshold보다 절댓값이 작은 가중치는 0으로 제거되고, 그 이상인 가중치만 보존된다. 설명의 편의를 위해 본 절에서는 $p=5$ 로 예를 들며, 이때 threshold는 32이므로 $|w| \leq 31$ 은 제거되고, $|w| \geq 32$ 만 남는다.

3.1.1.2 Log Quantization of Residual

두 번째 단계에서는 pruning 후 남은 non-zero 가중치에 대해 우선 residual을 계산한다.

$$r = |w| - 2^p, \quad (r \geq 0)$$

이 residual은 3bit 지수 z 로 Log Quantization 된다.

$$z = \begin{cases} 0, & r = 0 \\ \lfloor \log_2 |r| \rfloor, & r \neq 0 \end{cases}$$

여기서 z 를 3bit으로 제한한 이유는 Baseline 네트워크가 사용하는 INT8 가중치의 범위가 $[-128, 127]$ 이기 때문이다. Pruning exponent p 가 작아질수록 양자화해야 할 residual의 범위는 넓어진다. $p=0$ 인 최악의 경우 residual은 $[-128, 127]$ 범위에 분포한다. 이를 로그 도메인으로 변환하면 z 값은 항상 $[0, 7]$ 범위 안에 포함되므로, 3bit 지수 z 로 충분히 표현할 수 있다. 또한 해당 범위를 벗어나는 weight는 존재하지 않기 때문에, 기존 Log Quantization 식에 포함된 Clipping 연산은 필요하지 않다. 부호 비트 s 의 결정은 기존의 Log Quantization과 동일한 방식으로 수행된다. 즉, 가중치가 0 이상이면 $s=0$, 0 미만이면 $s=1$ 로 정의된다. 최종 가중치 표현은

$$\tilde{w} = \{s, z\}$$

와 같이 부호 비트 s 와 지수 z 의 쌍으로 나타난다.

앞서 설명한 Log Quantization of Residual은 pruning 후 남은 non-zero 가중치에만 적용한다. Pruning을 통해 제거된 가중치($w=0$)는 별도의 방식으로 표현한다. INT8 가중치에서 1bit 부호 s 와 3bit 지수 z 로 표현해야 할 residual의 최대 범위는 $p=0$ 인 경우의 $[-128, 127]$ 이다. 즉, residual이 128이 되는 경우는 존재하지 않으므로, 해당 경우의 quantized 표현인 $\tilde{w}=0111$ 은 사용되지 않는다. 그러므로 $\tilde{w}=0111$ 을 pruning 된 zero 가중치를 나타내는 표현으로 할당하였다. 이를 통해 메모리 및 연산 과정에서 zero와 non-zero 가중치를 구분할 수 있다. P3LQR을 적용한 몇 가지 예시를 [표 1]에 나타내었다.

w	residual	z	s	$\tilde{w}$
-5				0111
32	0	0	0	0000
-35	3	1	1	1001
56	24	4	0	0100

[표 1] $p = 5$ 일 때 P3LQR을 적용한 예시

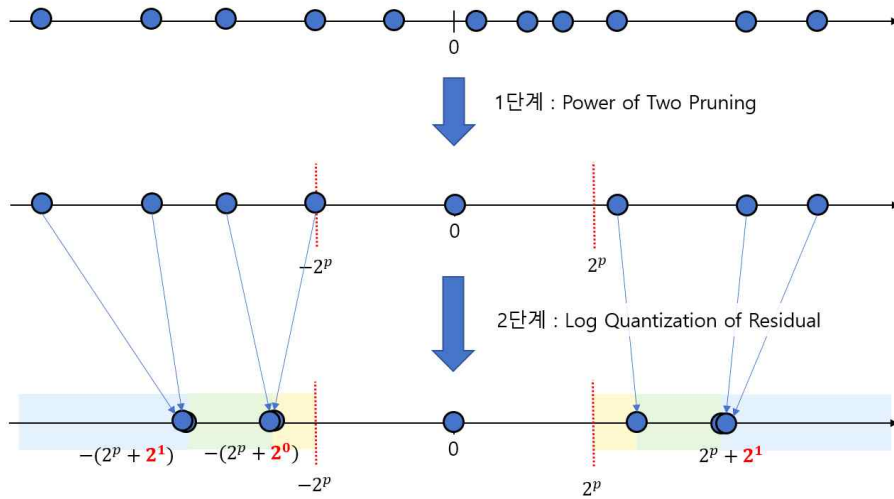
P3LQR을 적용한 후의 가중치는

$$\hat{w} = (-1)^s \cdot (2^p + 2^z)$$

로 복원된다. 이를 MAC 연산에 적용하면

$$y \approx \sum_{i=1}^n \hat{w}_i \cdot x_i = \sum_{i=1}^n \{(-1)^{s_i} \cdot (2^p + 2^{z_i})\} \cdot x_i = \sum_{i=1}^n (-1)^{s_i} \cdot \{(x_i \ll p) + (x_i \ll z_i)\}$$

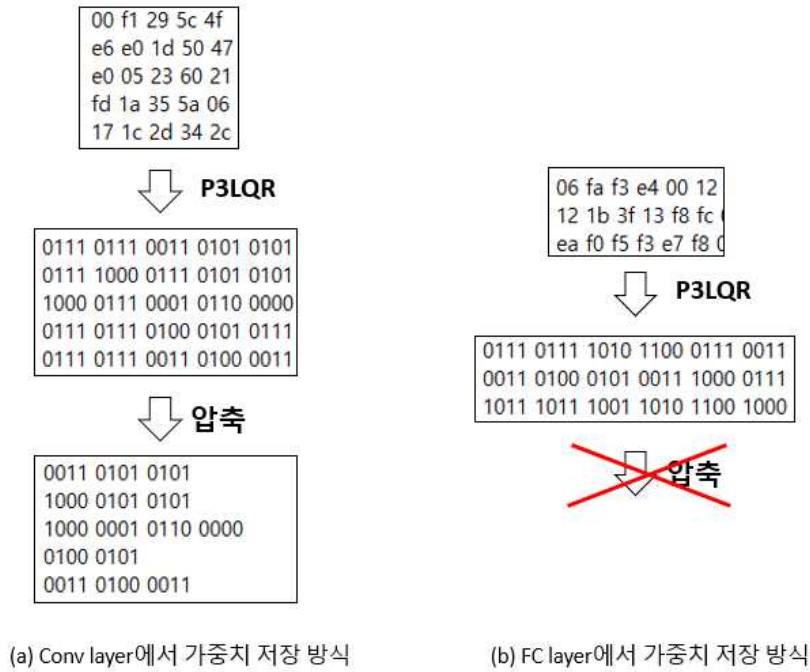
와 같이 표현되므로, 곱셈 연산이 두 번의 shift와 한 번의 덧셈으로 단순화된다. 곱셈 연산이 한 번의 shift로 대체되는 기존의 Log Quantization 방식보다는 복잡하지만, 모델의 정확도를 해치지 않으면서도 pruning을 동시에 적용할 수 있기 때문에 기존 Log Quantization 방식보다 오히려 더 큰 PPA 개선 효과를 얻을 수 있다. 이러한 P3LQR의 전체 과정을 정리한 도식은 [그림 7]에 나타나 있다.



[그림 7] P3LQR의 전체 과정

3.1.2 Hardware implementation of P3LQR

P3LQR을 적용함으로써 불필요한 가중치에 대한 곱셈 연산을 생략할 수 있으며, 남은 곱셈 연산 역시 두 번의 Shift와 한 번의 덧셈으로 구성된 간단한 Shift 모듈로 대체하여 전력 소모와 면적을 절감할 수 있다. 특히 Convolution layer에서는 각 Shift 모듈이 하나의 가중치와 일대일로 대응하므로, Pruning 단계에서 제거된 가중치에 대응하는 Shift 모듈 자체를 회로에서 제거할 수 있다. 회로 수준에서 zero 가중치에 대한 연산을 생략하므로, zero 가중치는 메모리에 저장할 필요가 없으며, pruning 후 남은 non-zero 가중치만 압축하여 저장하면 된다. 이와 같은 Convolution layer에서의 가중치 저장 방식을 [그림 8] (a)에 나타내었다.



[그림 8] Convolution layer와 FC layer의 가중치 저장 방식

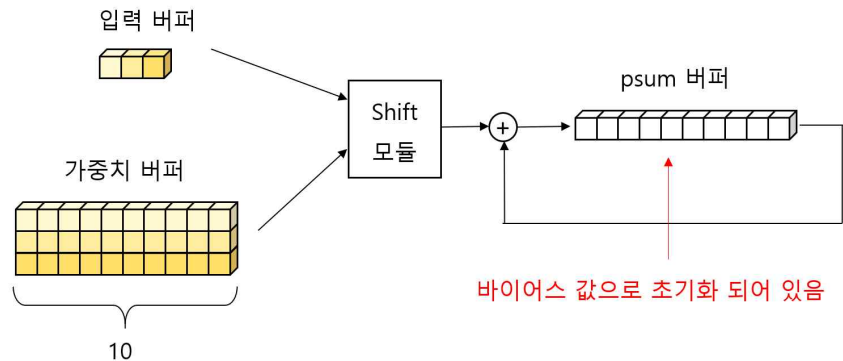
반면 FC layer에서는 각 Shift 모듈이 하나의 가중치와 일대일로 대응하지 않는다. 다시 말해, 각 Shift 모듈이 여러 개의 가중치를 처리하는 구조이다. 따라서 일부 가중치가 pruning 되더라도 Shift 모듈을 제거할 수 없다. 이 경우 pruning된 가중치도 메모리에 저장해야 하므로 Convolution layer에서와 같은 압축 저장 방식은 사용할 수 없다. 대신, 앞서 P3LQR 알고리즘 설명 단계에서 제시한 것처럼 pruning된 가중치를 $\tilde{w}=0111$ 로 메모리에 저장한다. FC layer에서의 가중치를 저장하는 방식은 [그림 8] (b)에 나타내었다.

3.1.3 Streaming FC Architecture

Baseline의 FC layer는 Maxpool_relu_2 모듈로부터 $1 \times 1 \times 3$ 크기의 입력을 4 cycle마다 받으며, 총 16 번의 입력을 받아 전체 48개의 입력이 내부 버퍼에 저장된다. 모든 입력이 들어온 후, 순차적인 10번의 MAC 연산을 통해 10개의 출력을 생성하는 구조이다. 그러나 이러한 구조는 모든 입력을 내부 버퍼에 저장해야 하기 때문에 메모리 사용량이 크다는 단점이 있다. 그리고 모든 입력(48개)이 들어온 후 연산을 시작하기 때문에 latency가 크다. 또한 valid_in 신호가 들어올 때만 출력이 발생하도록 설계되어 있어, 출력이 매 사이클마다 나오지 않고, valid_in 신호가 인가될 때만 나온다. 이러한 동작 방식은 불필요한 대기를 초래하여 전체 latency를 크게 증가시키는 원인이 된다.

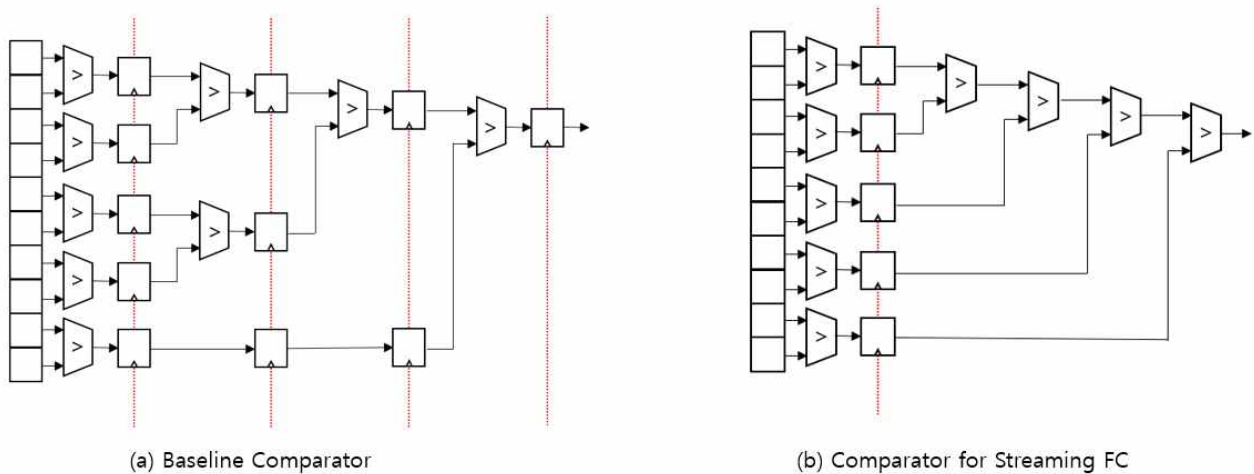
이러한 문제를 해결하기 위해 본 프로젝트에서는 FC layer를 입력 스트리밍 기반 구조로 최적화하였다. Maxpool_relu_2 layer에서 $1 \times 1 \times 3$ 크기의 입력이 들어올 때마다, 해당 3개의 입력과 30개의 가중치(3×10)를 곱하여 10개의 부분합을 계산하고, 이를 크기 10인 psum 버퍼에 누적한다. psum 버퍼의 초기값은 각 출력에 해당하는 바이어스로 설정하여, 연산 종료 시 별도로 바이어스를 더할 필요가 없도록 하였다. 한 입력에 대한 연산이 끝나면 가중치 버퍼를 다음 가중치로 업데이트하여, 이후 입력이 들

어왔을 때 바로 새로운 연산에 활용할 수 있도록 한다. 이렇게 하여 16번의 입력이 모두 들어온 바로 다음 사이클에 10개의 출력이 동시에 완성되어 comparator로 전달된다. 최적화된 FC layer의 구조는 [그림 9]에 나타내었다.



[그림 9] Streaming FC Architecture

메모리 사용량 관점에서, 최적화한 FC layer는 3개의 입력 버퍼, 30개의 가중치 버퍼 그리고 10개의 psum 버퍼로 구성된다. 따라서 총 43개의 버퍼만 필요하며, 이는 Baseline의 528개 (48개의 입력 버퍼 + 480개의 가중치 버퍼) 대비 약 92% 감소한 수치이다. 연산 자원 측면에서도 Baseline이 48개의 병렬 곱셈기를 필요로 하는 반면, 최적화한 FC layer는 30개의 곱셈기만 병렬로 동작하면 되므로 전력 소모와 면적을 줄일 수 있다. 또한 출력은 16번째 입력이 들어온 직후 1 cycle 만에 10개가 동시에 발생하고, valid_in 신호에 종속되지 않으므로 전체 latency 역시 크게 단축된다.



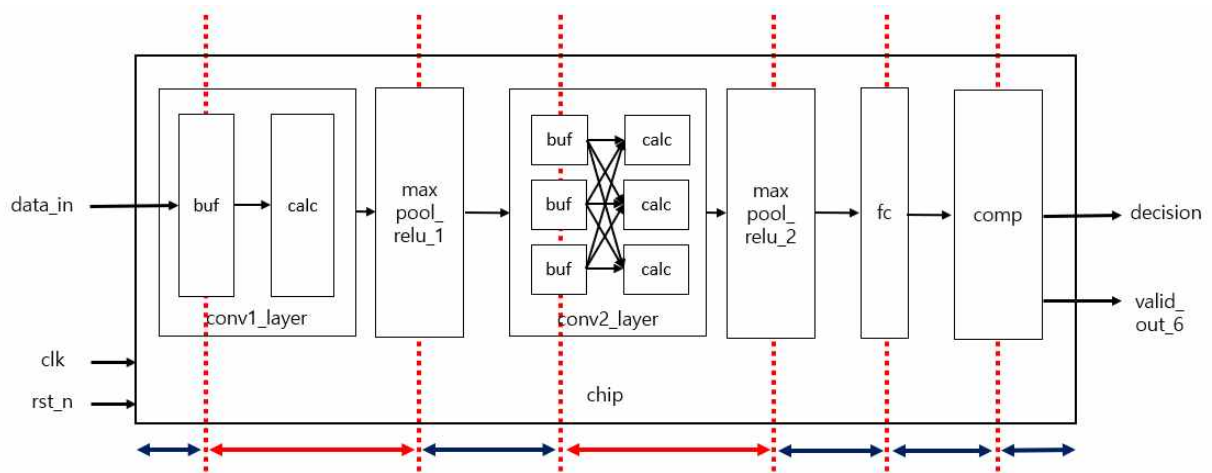
[그림 10] Comparator Architecture

Streaming FC Architecture에 맞게 Comparator의 구조도 최적화하였다. Baseline의 Comparator는 순차적으로 들어오는 FC layer의 출력을 내부 버퍼에 저장하고, 버퍼가 가득 차면 비교 연산을 시작한다. 비교 연산은 [그림 10] (a)와 같이 단계적으로 수행된다. 그러나 Streaming FC Architecture를 적용하면 10개의 출력이 동시에 발생하므로, Comparator 로직을 수정하여 10개의 입력이 동시에 들어오는 즉시 비교 연산을 시작하도록 하였다. 또한, [그림 10] (b)와 같이 10개의 값에 대한 비교가 두 단계에 걸쳐 이루어지도록 설계하여, 비교에 필요한 cycle을 Baseline보다 줄였다.

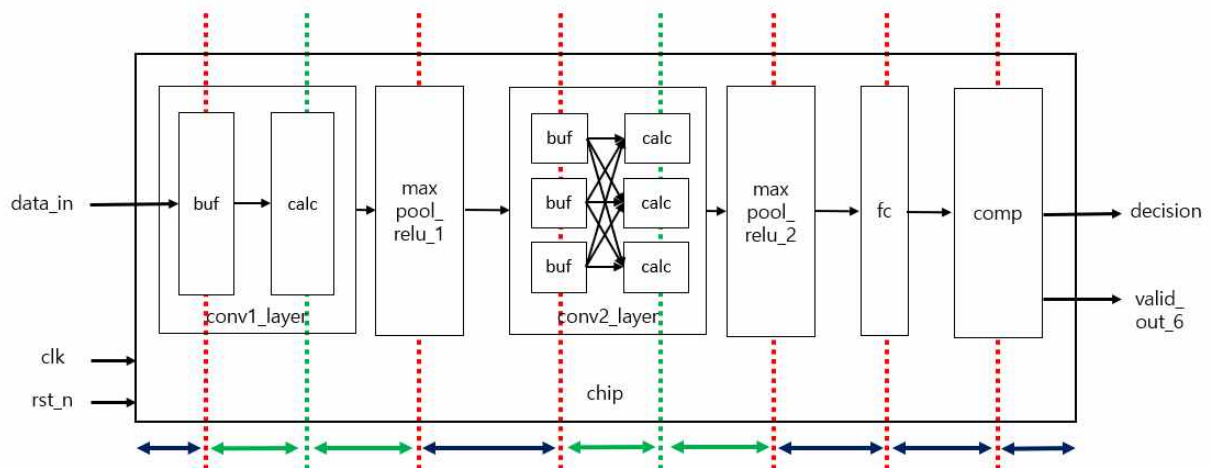
3.1.4 Convolution Layer Pipelining

파이프라인은 하나의 큰 연산을 여러 단계로 나누고, 각 단계를 병렬적으로 처리하는 기법이다. 파이프라인을 적용하면 각 단계를 처리하는 데 걸리는 시간이 줄어들고, 여러 단계를 동시에 실행할 수 있으므로 throughput이 개선된다. 전체 가속기의 동작 주파수를 결정하는 가장 느린 모듈을 파이프라이닝하면, 동작 주파수를 높여 전체 추론 시간을 단축할 수 있다.

Baseline의 conv1_calc와 conv2_calc 모듈은 MAC 연산을 수행하는 모듈로, 한 클럭 내에 여러 번의 곱셈과 덧셈 연산을 처리해야 하므로 latency가 매우 크다. 따라서 두 모듈의 MAC 연산 경로를 두 stage로 분할하고 파이프라이닝하여 동작 주파수를 높이고 추론 시간을 단축하였다.



(a) Baseline Architecture



(b) Convolution Layer Pipelined Architecture

[그림 11] Baseline Architecture and Convolution Layer Pipelined Architecture

파이프라이닝 구조 설계 시, 파이프라인 레지스터의 개수를 최소화하는 동시에 각 stage 간의 연산 지연이 균형을 이루도록 구성하였다. 예를 들어, 5×5 커널을 사용하여 25개의 MAC 연산을 해야 하는 conv1_calc 모듈이 pruning을 통해 15개의 MAC 연산만 필요해진 경우, Stage 1에서는 4, 4, 4, 3개의

부분합을 계산하여 4개의 파이프라인 레지스터에 저장하고, Stage 2에서 이 4개의 값을 더해 최종 출력을 생성하도록 설계하였다. 마찬가지로 conv2_calc 모듈에서 pruning 후 33개의 MAC 연산이 필요한 경우에는 Stage 1에서 6, 6, 6, 5, 5, 5개의 부분합을 계산하고, Stage 2에서 6개의 파이프라인 레지스터 값을 합산하는 방식으로 구현하였다. 이와 같이 Convolution layer의 덧셈 트리를 균형적으로 분할함으로써 stage 간 연산 지연을 균등화하고, 필요한 파이프라인 레지스터 수를 최소화하여 높은 동작 주파수와 자원 효율을 동시에 달성하였다.

3.2 구현 결과

본 프로젝트에서 제안한 P3LQR 기법, Streaming FC Architecture, 및 Convolution Layer Pipelining의 효과를 정량적으로 검증하기 위하여, 다음 절차로 실험을 진행하였다. 먼저 각 layer별 pruning exponent p 를 탐색하였다. 정확도 저하를 최소화하면서 가능한 많은 가중치를 제거하기 위해 후보 p 값을 스캔하고, Baseline 디자인과 함께 주어진 1000장의 입력 이미지를 대상으로 정확도를 측정하는 top_tb_1000 테스트벤치로 평가하여 각 layer의 최적 p 를 결정하였다.

이후 결정된 p 를 고정한 상태에서 최적화 요소를 단계적으로 반영한 RTL 코드를 구현하였다: (a) Power-of-Two Parameter Pruning → (b) Log Quantization of Residual → (c) Streaming FC Architecture → (d) Convolution Layer Pipelining (최종 제안 구조). 각 단계의 RTL 코드는 Verilog HDL로 작성하였고, OpenROAD EDA Tool을 사용하여 ASAP7 7nm finFET 공정 노드에서 합성하였다. 합성된 넷리스트는 다시 한번 Vivado 시뮬레이터와 top_tb_1000 테스트벤치를 사용하여 올바르게 동작하는지 검증하였다. 또한, OpenROAD Tool에서 생성된 Post-synthesis 보고서로부터 Power, Area, Timing 지표를 추출하여 Baseline 대비 성능 개선 정도를 정량적으로 분석하였다.

3.2.1 Baseline

먼저, Baseline 모델의 동작을 top_tb_1000 테스트벤치를 통해 검증한 결과, 1000장의 이미지에 대해 97%의 정확도를 보였다([그림 12]). 이후 해당 모델을 합성하고 시뮬레이션을 진행하였으며, 그 결과를 [표 2], [그림 13]에 정리하였다. 타이밍 분석 결과, Clock period 2500ps와 2100ps에서는 positive worst slack을 보였으나, 2000ps에서는 -59.01ps로 negative worst slack이 발생하여 timing violation이 확인되었다([표 2]). 따라서 Baseline의 동작 주파수는 Clock period 2100ps에 해당하는 476MHz로 결정하였다. 또한 Worst slack은 FC layer에서 발생하는 것도 확인할 수 있다([그림 13] (a)).

```
995th input image : original value = 4, decision = 4 at 13306555 ps ==> Success
996th input image : original value = 5, decision = 5 at 13319915 ps ==> Success
997th input image : original value = 6, decision = 6 at 13333275 ps ==> Success
998th input image : original value = 7, decision = 7 at 13346635 ps ==> Success
999th input image : original value = 8, decision = 8 at 13359995 ps ==> Success
1000th input image : original value = 9, decision = 9 at 13373355 ps ==> Success

----- Final Accuracy for 1000 Input Image -----
Checked Images: 1000 / 1000
Correct Answers: 970 / 1000
Accuracy : 97.0%
run: Time (s): cpu = 00:00:10 ; elapsed = 00:00:14 . Memory (MB): peak = 1039.516 ; gain = 14.215
xsim: Time (s): cpu = 00:00:11 ; elapsed = 00:00:15 . Memory (MB): peak = 1039.516 ; gain = 121.199
```

[그림 12] Baseline 모델의 top_tb_1000 시뮬레이션 결과

clock period	worst slack
2500ps	440.99ps
2100ps	40.99ps
2000ps	-59.01ps

[표 2] Baseline의 worst slack

Post synthesis report_checks -path_delay max

Startpoint: fully_connected.out_idx[1]\$_SDFFE_PN0P_
(rising edge-triggered flip-flop clocked by clk)
Endpoint: comparator.buffer[9][11]\$_DFFE_PP_
(rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

(a) Worst slack path

Chip area for module 'Wchip': 24847.950420
of which used for sequential elements: 1773.511200 (7.14%)

(b) Area

Post synthesis report_power

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	5.37e-03	1.34e-04	8.83e-07	5.51e-03	1.6%
Combinational	1.27e-01	2.03e-01	1.90e-05	3.30e-01	98.4%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	1.32e-01	2.03e-01	1.99e-05	3.35e-01	100.0%
	39.4%	60.6%	0.0%		

(c) Power

[그림 13] Baseline Post-synthesis report

476MHz에서 전력 분석 결과, 총 전력 소모는 335mW로 나타났다([그림 13] (c)). 이를 바탕으로 전력 효율을 평가하기 위해 TOPS/W를 계산하였다. Baseline 모델에서 MAC 연산의 개수는 conv1_layer에서 $25 \times 3 = 75$ 개, conv2_layer에서 $25 \times 9 = 225$ 개, fc layer에서 48개로 총 348개이다. 이를 1장에서 소개한 TOPS/W 계산식에 대입하면,

$$TOPS/W = \frac{2 \times 348 \times 476 \times 10^6}{10^{12} \times 0.335} = 0.989$$

로 계산된다. 또한 합성 후 면적은 총 $24,8648\mu m^2$ 로 확인되었다([그림 13] (b)). 이와 같은 Baseline의 합성 결과들은 제안한 최적화 기법들의 성능 개선 효과를 평가하기 위한 기준으로 활용된다.

3.2.2 Layer별 Pruning exponent 탐색

P3LQR을 적용하기 위해 각 layer별로 적합한 Pruning exponent p 를 탐색하였다. 목표는 정확도를 유지하면서 최대한 많은 가중치를 제거하는 p 조합을 찾는 것이다. 먼저 각 layer에 대해 p 를 1부터 6까지 변화시키며 top_tb_1000 테스트벤치에서의 정확도를 측정하였다. 이때 한 번에 하나의 layer에만 P3LQR을 적용하고, 나머지 layer에는 적용하지 않는 방식으로 각 layer별 pruning 민감도를 확인하였다. 실험 결과는 [표 3]에 나타내었다. conv1_layer의 경우, $p = 1 \sim 5$ 구간에서는 정확도가 안정적으로 유지되었으나, $p = 6$ 에서 급격히 성능이 저하되었다. conv2 layer는 $p = 1 \sim 4$ 구간까지는 Baseline과 유사한 정확도를 보였으나, $p = 5$ 이상에서는 큰 폭의 성능 저하가 발생하였다. FC layer는 $p = 1 \sim 2$ 구간까지는 정확도가 유지되었으나, $p = 3$ 부터 점차 감소하여 $p = 4$ 이상에서 성능이 크게 저하되었다.

p	conv1	conv2	FC
1	97	97	97
2	97	96	97
3	96	98	95
4	97	97	90
5	98	55	62
6	78	28	10

[표 3] Layer별 Pruning exponent에 따른 정확도 변화 (단위: %)

위의 결과를 바탕으로 적합한 $\{p_{conv1}, p_{conv2}, p_{fc}\}$ 조합 후보를 선정하여 정확도를 측정하였다([표 4]). 실험 결과 $\{p_{conv1}, p_{conv2}, p_{fc}\} = \{5, 4, 3\}$ 조합이 정확도를 Baseline과 동일한 97%로 유지하면서 최대한 많은 가중치를 제거할 수 있었기에, 해당 조합을 최종적으로 채택하였다. 이 조합을 적용했을 때 layer별 pruning된 가중치 개수는 [표 5]에 제시하였다. 주목할 만한 점은 conv1_layer의 첫 번째 출력 채널(conv1_weight_1)의 가중치 25개가 모두 제거되어, 해당 채널 자체를 제거할 수 있다는 점이다. 이후의 설계 과정에서도 이를 반영하여 conv1_layer의 출력과 conv2_layer의 입력을 두 채널로 설계하였다.

$\{p_{conv1}, p_{conv2}, p_{fc}\}$	정확도[%]
{4, 3, 2}	97
{5, 3, 2}	98
{5, 4, 2}	98
{5, 4, 3}	97
{6, 4, 3}	82
{5, 5, 3}	53
{5, 4, 4}	89

[표 4] p 조합별 정확도

layer	weight	pruning된 가중치 개수
conv1	conv1_weight_1	25
	conv1_weight_2	10
	conv1_weight_3	14
conv2	conv2_weight_11	24
	conv2_weight_12	6
	conv2_weight_13	11
	conv2_weight_21	25
	conv2_weight_22	12
	conv2_weight_23	9
	conv2_weight_31	25
	conv2_weight_32	10
	conv2_weight_33	12
fc	fc_weight	115

[표 5] Layer별 pruning된 가중치 개수

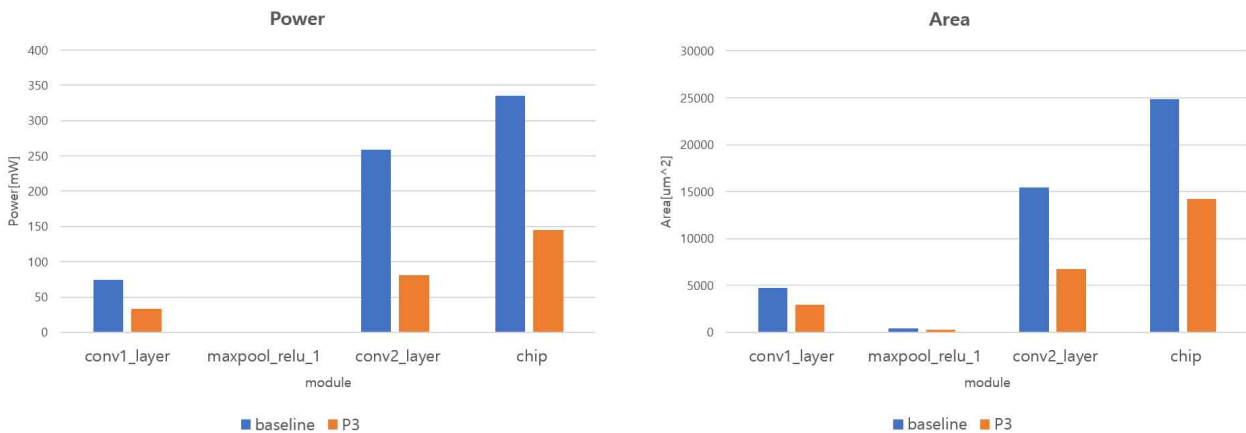
$\{p_{conv1}, p_{conv2}, p_{fc}\} = \{5, 4, 3\}$ 을 적용한 P3LQR을 메모리 자원 관점에서 분석하면 다음과 같다. 저장해야 할 파라미터 수는 기존 796개(가중치 780개 + 바이어스 16개)에서 612개(가중치 597개 + 바이어스 15개)로 감소하였다. 이는 conv1, conv2에서 pruning된 zero 가중치는 메모리에 저장하지 않으며, conv1의 출력 채널이 하나 줄어들었으므로 그에 해당하는 바이어스도 제거할 수 있기 때문이다. 더불어 Log Quantization of Residual 기법을 통해 기존의 8bit 가중치가 4bit로 양자화되었다. 결과적으로 전체 파라미터의 비트 수는 Baseline의 6,368bit에서 P3LQR 적용 후 2,508bit로 줄어들어, Baseline 대비 60.6% 감소하였다. 이는 메모리 자원 절감 측면에서 큰 이점을 제공한다.

3.2.3 Power-of-Two Parameter Pruning (P3)

$\{p_{conv1}, p_{conv2}, p_{fc}\} = \{5, 4, 3\}$ 을 고정한 상태에서 최적화 요소를 단계적으로 반영하여 설계를 진행하였다. 우선 P3LQR의 첫 번째 단계인 Power-of-Two Parameter Pruning만 적용하여 합성 및 시뮬레이션을 수행하였으며, 그 결과를 [표 6]과 [그림 14]로 정리하였다. Pruning의 영향을 직접적으로 받을 수 있는 모듈인 conv1_layer, maxpool_relu_1, conv2_layer의 결과와 탑 모듈인 chip 모듈의 결과를 함께 제시하였다. Worst slack과 Power는 Baseline과의 공평한 비교를 위해 Baseline에서 사용한 clock period인 2100ps에서 측정하였다. Power-of-Two Parameter Pruning을 적용한 결과, Baseline 대비 전력 소모는 약 56.7%, 면적은 약 42.9% 감소하였다.

module	worst slack[ps] (@ 2100ps)			power[mW] (@ 2100ps)			area[μm^2]		
	P3	baseline	변화량	P3	baseline	변화율	P3	baseline	변화율
conv1_layer	163.56	75.43	+88.13	33.2	74.3	-55.3%	2933	4701	-37.6%
maxpool_relu_1	1415.73	1369.7	+46.03	0.514	0.744	-30.9%	255	421	-39.4%
conv2_layer	648.02	512.6	+135.42	81.1	259	-68.7%	6705	15425	-56.5%
chip	40.99	40.99	0	145	335	-56.7%	14178	24848	-42.9%

[표 6] Power-of-Two Parameter Pruning 디자인 적용 후 합성 및 시뮬레이션 결과



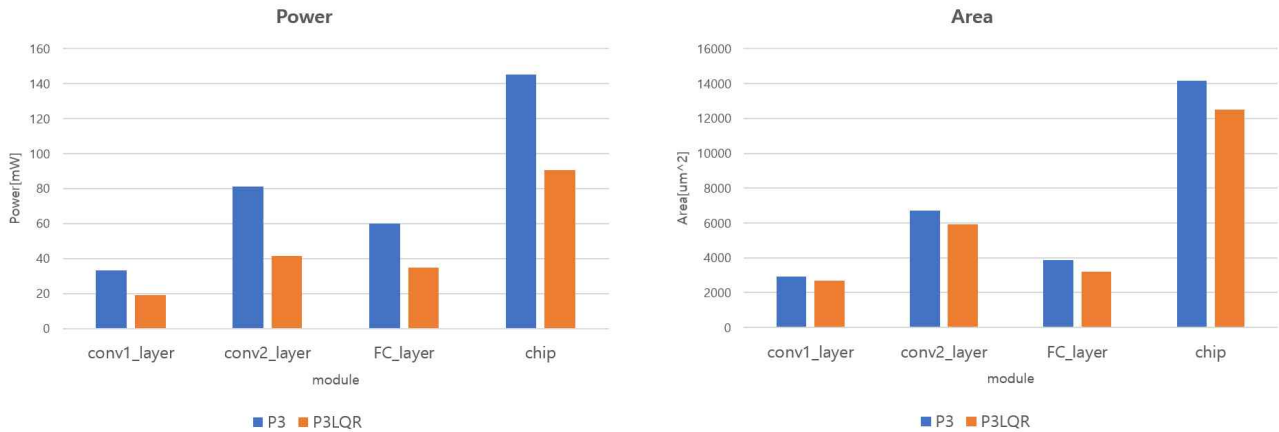
[그림 14] Power-of-Two Parameter Pruning 디자인 적용 후 합성 및 시뮬레이션 결과

3.2.4 P3LQR

다음으로 Pruning에 Log Quantization of Residual을 포함한 P3LQR을 반영하여 RTL을 설계하였다. Log Quantization of Residual을 적용함으로써 전력 소모와 면적이 큰 곱셈기를 두 번의 Shift와 한 번의 덧셈으로 구성된 Shift 모듈로 대체하였다. 합성 및 시뮬레이션 결과를 Pruning만 적용한 경우와 비교하여 [표 7]과 [그림 15]에 정리하였다. 실험은 Baseline과 동일하게 clock period 2100ps에서 진행하였다. Log Quantization of Residual을 적용함으로써 MAC 연산이 포함된 conv1_layer, conv2_layer, FC layer에서 전력 소모와 면적이 모두 감소하였다. 특히 Baseline에서 Worst Slack이 발생하던 FC layer의 타이밍이 크게 개선되어, 전체 모델의 worst slack은 기존 모델 및 Pruning만 적용한 모델의 40.99ps에서 351.86ps로 대폭 증가하였다. 또한 전력 소모는 Pruning만 적용한 경우에 비해 37.4% 감소하였으며, 면적은 11.9% 감소하였다.

module	worst slack[ps] (@ 2100ps)			power[mW] (@ 2100ps)			area[μm^2]		
	P3LQR	P3	변화량	P3LQR	P3	변화율	P3LQR	P3	변화율
conv1_layer	273.84	163.56	+110.28	19.2	33.2	-42.2%	2702	2933	-7.88%
conv2_layer	557.28	648.02	-90.74	41.4	81.1	-49.0%	5919	6705	-11.7%
FC layer	-1.53	-310.33	+308.8	34.9	60.1	-41.9%	3206	3874	-17.2%
chip	351.86	40.99	+310.87	90.7	145	-37.4%	12494	14178	-11.9%

[표 7] P3LQR 디자인 적용 후 합성 및 시뮬레이션 결과



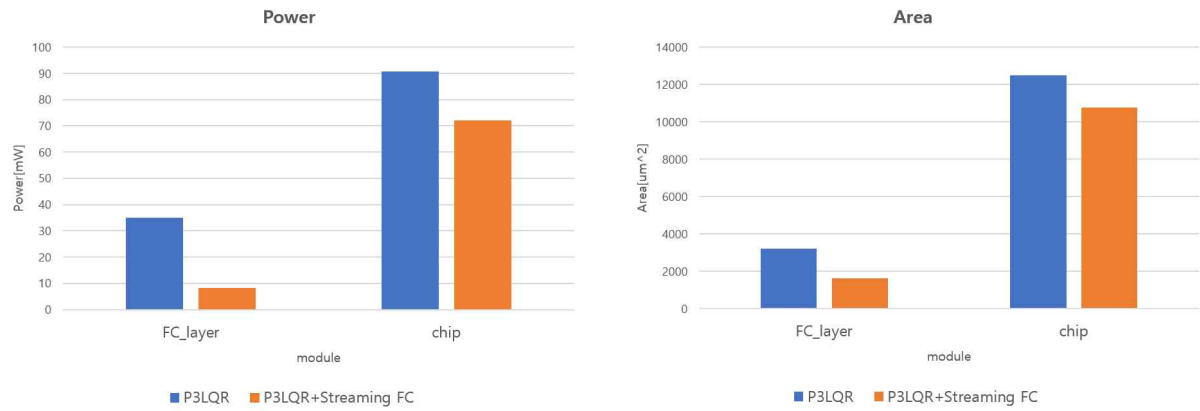
[그림 15] P3LQR 디자인 적용 후 합성 및 시뮬레이션 결과

3.2.5 Streaming FC Architecture

다음으로 FC layer를 streaming 기반 구조로 개선한 후, 이를 3.2.4의 결과와 비교하였다. 결과는 [표 8], [그림 16]에 정리하였다. FC layer의 worst slack, 전력 소모, 면적이 모두 큰 폭으로 개선되었다. 전체 하드웨어의 Worst slack은 92.71ps 증가하였고, 전력 소모는 20.4% 감소하였으며, 면적은 13.9% 감소하였다.

module	worst slack[ps] (@ 2100ps)			power[mW] (@ 2100ps)			area[μm^2]		
	P3LQR + Streaming FC	P3LQR	변화량	P3LQR + Streaming FC	P3LQR	변화율	P3LQR + Streaming FC	P3LQR	변화율
FC layer	922.72	-1.53	+924.25	8.2	34.9	-76.5%	1614	3206	-49.7%
chip	444.57	351.86	+92.71	72.2	90.7	-20.4%	10757	12494	-13.9%

[표 8] Streaming FC Architecture 합성 및 시뮬레이션 결과



[그림 16] Streaming FC Architecture 합성 및 시뮬레이션 결과

3.2.6 Convolution Layer Pipelining

P3LQR과 Streaming FC Architecture을 적용한 모델의 타이밍 분석 결과를 [표 9]에 나타내었다. 해당 모델은 1600ps까지 정상 동작하였으며, 1500ps부터는 -35.43ps의 negative worst slack이 발생하였다. 따라서 최대 동작 주파수는 625MHz(1600ps)이다.

clk period [ps]	worst slack [ps]
2100	444.57
1700	124.57
1600	44.57
1500	-35.43

[표 9] P3LQR과 Streaming FC를 적용한 모델의 worst slack

모듈	worst slack [ps]
conv1_layer	-26.16
maxpool_relu_1	1015.73
conv2_layer	157.28
maxpool_relu_2	1000.11
FC_layer	522.72
Comparator	601.48

[표 10] 모듈별 worst slack (@1600ps)

최대 동작 주파수에서 각 모듈의 worst slack을 분석한 결과, conv1_layer와 conv2_layer에서 가장 작은 worst slack이 발생하였다([표 10]). 전체 모델의 동작 주파수를 개선하기 위해 latency가 가장 큰 두 Convolution Layer에 대해 2-stage 파이프라이닝을 적용하였다. 그 결과, 모델의 최대 동작 주파수가 1GHz(1000ps)로 향상되었으며, 추론 시간은 기존 1.427 μ s에서 0.894 μ s로 감소하였다([표 11], [표 12]).

clk period [ps]	worst slack [ps]
1600	489.5
1100	89.5
1000	1.48
900	-98.52

[표 11] 파이프라이닝 적용한 모델의 worst slack

평가 지표	파이프라이닝 전	파이프라이닝 후
최소 clk period	1600ps	1000ps
최대 동작 주파수	625MHz	1GHz
# of cycles	892	894
추론 시간	1.427 μ s	0.894 μ s

[표 12] 파이프라이닝 전후 성능 비교

3.2.7 최종 성능 비교

제안한 최종 CNN 가속기(Proposed design)의 성능을 Baseline과 비교하였다. Proposed design의 netlist를 top_tb_1000 테스트벤치로 시뮬레이션한 결과([그림 17]), 1000장의 이미지에 대한 예측 정확도는 97%로 Baseline과 같게 유지되었다.

```

995th input image : original value = 4, decision = 4 at 8925145 ps ==> Success
996th input image : original value = 5, decision = 5 at 8934115 ps ==> Success
997th input image : original value = 6, decision = 6 at 8943085 ps ==> Success
998th input image : original value = 7, decision = 7 at 8952055 ps ==> Success
999th input image : original value = 8, decision = 8 at 8961025 ps ==> Success
1000th input image : original value = 9, decision = 9 at 8969995 ps ==> Success

----- Final Accuracy for 1000 Input Image -----
Checked Images: 1000 / 1000
Correct Answers: 970 / 1000
Accuracy : 97.0%
$stop called at time : 8970015 ps : File "C:/Users/USER/Desktop/KNU_IDEC/Vivado/FIN
run: Time (s): cpu = 00:16:51 ; elapsed = 02:03:03 ; Memory (MB): peak = 2957.031 ;
xsim: Time (s): cpu = 00:16:54 ; elapsed = 02:03:07 ; Memory (MB): peak = 2957.031

```

[그림 17] Proposed design의 top_tb_1000
시뮬레이션 결과

Post synthesis report_power					
Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	7.98e-03	2.65e-04	7.17e-07	8.24e-03	23.1%
Combinational	1.21e-02	1.53e-02	8.87e-06	2.74e-02	76.9%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	2.01e-02	1.56e-02	9.59e-06	3.57e-02	100.0%
	56.3%	43.6%	0.0%		

[그림 18] Proposed design의 Power (@1000ps)

Proposed design의 최대 동작 주파수 1GHz(1000ps)에서의 전력 소모는 35.7mW로 측정되었다([그림 18]). 이를 바탕으로 계산한 TOPS/W는

$$TOPS/W = \frac{2 \times 348 \times 1 \times 10^9}{10^{12} \times 0.0357} = 19.496$$

이며, 이는 Baseline의 0.989 대비 1871% 향상된 수치이다. 즉, 제안한 최적화 요소들이 TOPS/W 개선에 매우 효과적임을 알 수 있다.

추론 시간에 있어서도 Proposed design은 뛰어난 성능을 보였다. Baseline은 MNIST 이미지 하나를 처리하는 데, 총 1335 cycle ($2.80\mu s$)이 필요했으나, Proposed design은 894 cycle ($0.894\mu s$)로 추론 시간이 68.1% 단축되었다([표 13]). 이는 Streaming FC Architecture를 통한 cycle 감소와 동작 주파수 향상에 따른 효과이다.

평가 지표	Baseline	Proposed design
정확도	97%	97%
최대 동작 주파수	476MHz	1GHz
최대 동작 주파수에서의 전력	335mW	35.7mW
TOPS/W	0.989	19.496 (+1,871%)
# of cycle	1335	894
추론 시간	$2.80\mu s$	$0.894\mu s$
면적	$24848\mu m^2$	$11146\mu m^2$

[표 13] Baseline과 Proposed design의 성능 비교

Chip area for module 'Wchip': 11145.535200
of which used for sequential elements: 1381.892400 (12.40%)

[그림 19] Proposed design의 면적

마지막으로, 합성 후 면적도 큰 차이를 보였다. Baseline 모델의 전체 면적은 $24848\mu m^2$ 였던 반면, Proposed design은 $11146\mu m^2$ 로 줄어들어 면적이 55.1% 감소하였다([표 13], [그림 19]).

종합하면, P3LQR, Streaming FC Architecture, 그리고 Convolution Layer Pipelining을 반영한 Proposed design은 인식 정확도를 97%로 유지하면서도 TOPS/W 개선 (+1871%), 추론 시간 단축 (-68.1%), 면적 감소 (-55.1%)에 이르는 모든 PPA 지표의 향상을 달성하였다.

4. 결과 및 기대효과

본 프로젝트에서는 MNIST 데이터셋을 분류하는 CNN 가속기의 성능 향상을 위해 P3LQR, Streaming FC Architecture, Convolution Layer Pipelining을 단계적으로 적용하였다. 먼저, Pruning과 Log Quantization을 효과적으로 결합한 P3LQR 알고리즘으로 모델을 경량화하여 메모리 용량을 줄이고, 곱셈기를 Shift 모듈로 대체함으로써 연산 효율을 크게 개선하였다. 다음으로, Baseline 모델의 비효율적인 FC layer를 Streaming FC Architecture로 변경하여 전력 및 면적 효율을 개선하는 동시에 추론 속도를 향상하였다. 마지막으로, 앞선 두 최적화 요소를 반영한 모델의 타이밍을 분석하고, worst slack이 가장 작은 Convolution Layer를 2-stage로 파이프라이닝 함으로써, 전체 모델의 동작 주파수를 높이고 추론 시간을 단축하였다.

이러한 최적화 기법을 적용한 결과, Proposed design은 정확도를 Baseline과 같은 97%로 유지하면서, 최대 동작 주파수가 476MHz에서 1GHz로 향상되었다. 동시에 추론에 필요한 cycle이 1335 cycle에서 894 cycle로 감소하여, 추론 시간은 $2.80\mu s$ 에서 $0.894\mu s$ 로 68.1% 단축되었다. TOPS/W는 0.989에서 19.496으로 1871% 증가하였으며, 면적 역시 $24848\mu m^2$ 에서 $11146\mu m^2$ 로 55.1% 감소하였다. 결과적으로 Proposed design은 Baseline 대비 모든 PPA 지표에서 뚜렷한 개선을 보였으며, 특히 TOPS/W에서 탁월한 성능을 보여주는 에너지 효율적인 CNN 가속기임이 입증되었다.

이러한 결과를 바탕으로, 본 프로젝트에서 제안한 최적화 요소들은 향후 다양한 딥러닝 하드웨어 설계에 폭넓게 응용될 수 있는 가능성을 지닌다. P3LQR 알고리즘은 모델 크기를 줄이기 위한 Pruning과, MAC 연산 효율을 높이기 위한 Log Quantization을 동시에 적용할 수 있는 새로운 방법을 제안하였으며, 두 기법을 동시에 적용할 때 발생하는 정확도 저하 문제를 효과적으로 해결하였다. 이러한 경량화 알고리즘은 본 프로젝트에서 사용된 소형 CNN뿐만 아니라 ResNet, VGG 등 대규모 CNN으로 확장할 수 있을 것으로 기대된다.

또한 Streaming FC Architecture는 모든 입력이 도착할 때까지 대기해야 하는 기존 구조의 한계를 극복하고 입력 데이터를 실시간으로 처리함으로써 불필요한 내부 버퍼를 줄이고 처리 속도를 높이는 구조적 장점을 제공한다. 마지막으로 각 모듈의 타이밍을 분석하여 파이프라이닝을 적용하는 방법은 CNN

가속기에 국한되지 않고, 다양한 하드웨어 설계 전반에서 동작 주파수 향상과 추론 시간 단축을 달성할 수 있는 보편적인 최적화 전략으로 활용될 수 있다.

따라서 본 프로젝트에서 제안한 최적화 기법은 고속화, 저전력화, 소형화를 동시에 달성할 수 있는 효율적인 딥러닝 가속기 설계 방향을 제시하며, 차세대 AI 하드웨어 구현에 실질적인 기여를 할 수 있을 것으로 기대된다.

5. Reference

- [1] Han, Song, et al. "Learning both weights and connections for efficient neural network." Advances in neural information processing systems 28 (2015).
- [2] Miyashita, Daisuke, Edward H. Lee, and Boris Murmann. "Convolutional neural networks using logarithmic data representation." arXiv preprint arXiv:1603.01025 (2016).
- [3] Han, Song, Huizi Mao, and William J. Dally. "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding." arXiv preprint arXiv:1510.00149 (2015).